Harnessing Insights from Streams: Unlocking Real-Time Data Flow with Docker and Cassandra in the Apache Ecosystem

Jay Oza
Computer Engineering
K.J. Somaiya Institute of Technology
jay.oza@somaiya.edu

Prof. Abhijit Patil
Computer Engineering
K.J. Somaiya Institute of Technology
abhijit.patil@somaiya.edu

Chirag Maniyath
Computer Engineering
K.J. Somaiya Institute of Technology
chirag.maniyath@somaiya.edu

Rishi More
Computer Engineering
K.J. Somaiya Institute of Technology
rishi.vm@somaiya.edu

Gitesh Kambli
Computer Engineering
K.J. Somaiya Institute of Technology
gitesh.kambli@somaiya.edu

Amit Maity
Computer Engineering
K.J. Somaiya Institute of Technology
amit.maity@somaiya.edu

2024 IEEE Recent Advances in Intelligent Computational Systems (RAICS) | 979-8-3503-8168-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/RAICS61201.2024.10690115

***Abstract*—Real-time data streaming pipelines are immensely valuable in today's data-driven world since they enable continuous data processing and analytics. This research paper provides a comprehensive exploration of the architecture, development, and deployment of an advanced real-time data streaming pipeline. It utilizes Docker for containerization, Apache Kafka for distributed streaming, Apache Spark for dynamic data transformation, and Cassandra for efficient NoSQL storage. The study outlines the intricacies of integrating these technologies, examining the pipeline's components, functionalities, performance metrics, and potential applications. Through this case study, the paper showcases the efficacy of open-source tools in constructing highly scalable and resilient data streaming pipelines.**

***Keywords*—big data, data analytics, data streaming, Kafka, Docker, Cassandra, Spark**

I. Introduction

With the continuous creation of massive datasets in today's digital world, there is a critical need for advanced real-time data processing capabilities. The widespread use of the Internet of Things, social media, and digital interactions is generating data at an unprecedented rate. To tap into the valuable insights this data offers, organizations are adopting real-time streaming pipelines. These pipelines ingest, process, and store data as it is produced to empower timely, data-driven decision-making. This research paper comprehensively investigates building a sophisticated real-time streaming pipeline, mapping out the process from data intake through storage. It provides an in-depth examination of creating a robust pipeline to handle the immense data volumes in the modern landscape.

This research focuses on integrating leading open-source technologies to build a robust, scalable real-time data streaming pipeline. Specifically, it strategically combines Docker, Apache Kafka, Apache Spark, and Cassandra to leverage their unique strengths. Docker provides portability and reliability through containerized microservices. Kafka enables high-throughput, distributed, fault-tolerant data streaming. Spark performs real-time data processing with its powerful engine. Cassandra offers a flexible, scalable NoSQL data store for the processed pipeline data. Together, these technologies enable a cohesive integration of containerization, streaming, processing, and storage to create a resilient real-time pipeline. Since these are open-source tools, they facilitate scalability and agility within the pipeline architecture. In summary, this project represents an optimal combination of modern technologies for real-time data pipelines.

The aim of this project is an in-depth guide for creating a full data engineering solution, not merely a basic pipeline. It describes carefully designing the architecture from ingestion to Cassandra storage. Apache Airflow orchestrates and smoothes data flow through each pipeline phase. In addition to core technologies like Kafka, Spark, and Cassandra, it utilizes Python, PostgreSQL, and Docker. Instead of an isolated pipeline, it takes a holistic approach to building an end-to-end system, with Airflow coordinating components. The motivation is to provide a detailed reference for constructing real-world, robust data solutions.

Airflow, Python, Kafka, Spark, Cassandra, and Docker were strategically chosen to construct a flexible, robust toolset for real-world data challenges. While a custom API provides foundational data, Kaggle datasets - user purchases, IoT, Twitter - evaluate real-world pipeline relevance. Testing on public Kaggle data beyond the API ensures the solution generalizes across diverse real-world scenarios. The tech selections reflect efforts to build a versatile pipeline. Including Kaggle data injects vital real-world applicability into testing and validation. This comprehensive approach strengthens the pipeline's adaptability to varied organizational data challenges.

This research aims to not only showcase the capabilities of the technologies used but also provide a practical guide for data engineers working with real-time streaming pipelines.

The following sections will thoroughly examine the system architecture, methodology, results, analysis, and future work to impart comprehensive knowledge on designing, constructing, and improving the pipeline. By exploring the intricacies of implementation, performance, and growth opportunities, this paper goes beyond demonstrating technical skills. The goal is to offer an actionable resource for organizations and professionals seeking to leverage real-time streaming for impactful insights and data-driven decisions. The research strives to make meaningful contributions to the evolving field of data engineering, enabling the creation of robust real-time pipelines that fully harness the value of streaming data.

II. Literature Survey

Optimization techniques for enhancing the efficiency of Apache Spark applications for large data analytics are covered in the study by Gupta et al. [1]. It looks at methods such as optimizing Spark executor setups, repartitioning data to boost parallelism, employing broadcast variables to efficiently disseminate data to all nodes, and caching frequently used DataFrames into memory or disk. The methods are assessed within the framework of a PySpark-built content recommender system. Combining several optimization techniques, including caching, broadcast joins, repartitioning with more cores, and optimizing the number of executors per node and their allotted memory, results in considerable savings in processing time.

The paper by Peddireddy [2] proposes using Apache Kafka [3], an open-source distributed streaming platform, to facilitate real-time data processing, alerting, and reporting for enterprises. The paper explores Kafka's scalable and fault-tolerant architecture for building data pipelines and streaming applications. It discusses ingesting enterprise data streams using Kafka Connect, processing them with Kafka Streams, storing processed data for reporting, generating real-time alerts based on conditions, and monitoring Kafka clusters. Integration capabilities with external systems are highlighted. Use cases of leveraging Kafka streams in industries like financial services, healthcare, transportation, and retail are covered.

The paper by Shree et al. [4] analyzes Kafka as a distributed, scalable platform for big data stream management and processing. Kafka's publish-subscribe messaging system, use of persisted logs, partitioning, parallelism, and fault tolerance make it suitable for low latency and high throughput data pipelines. Four Kafka APIs cater to messaging, storage, stream processing, and connector integration needs. Benefits highlighted include strong ordering guarantees, replayable streams, and unification of messaging, storage, and processing approaches. Comparison with Flume brings out Kafka's pull-based approach. Use cases like website activity tracking, log aggregation, commit logs and messaging are discussed.

Drohobytskiy et al. [5] demonstrate customizing Kafka stream processing using Spark Structured Streaming. Conditional monitoring procedures to process irregular data streams efficiently are shown. Techniques for managing Kafka offsets during spark streaming to avoid repeat reads are explained. Checkpointing limitations are highlighted and alternative offset storage options like databases are suggested. The development uses case-class modeling and streaming query listeners. The solution enables scheduled spark streaming jobs to restart reading Kafka topics from last stopped offsets.

The research paper titled "A Study of Apache Kafka in Big Data Stream Processing" [6] offers a comprehensive exploration of Apache Kafka's benefits and applications in real-time data analytics. Emphasizing the significance of stream processing in today's data-driven landscape, the authors advocate for stream processing systems, utilizing Stream Processing Elements (SPE) to analyze data before storage. Apache Kafka is introduced as a scalable, distributed, and reliable architecture with features akin to messaging systems, facilitating high-volume data handling. The paper delves into Kafka's framework, detailing key terminologies such as topics, producers, consumers, connectors, stream processors, brokers, and Zookeepers, underscoring its scalability, reliability, data transformation capabilities, and low latency. Contrasting with traditional messaging systems like IBM WebSphere MQ and JSM, along with newer systems like Hedwig, the authors assert Kafka's superiority in scalability, reliability, and durability. The paper cites successful implementations of Kafka by companies such as Twitter, LinkedIn, Yahoo!, and Netflix in real-time data analytics, contributing significantly to stream processing literature and highlighting Apache Kafka's pivotal role in big data stream processing.

The research paper "Real-Time Detection of Social Bots on Twitter Using Machine Learning and Apache Kafka" [7] addresses the pressing issue of social bots infiltrating online social networks, particularly Twitter. Social bots, automated accounts controlled by scripts, pose a significant threat by disseminating misinformation, spreading rumors, and manipulating trending content. Traditional offline bot detection techniques have limitations, as they do not facilitate real-time detection during the incident. To address this gap, the paper proposes a real-time detection framework, Stream Bot Detection (SBD), which combines Twitter API for data collection, Apache Kafka for streaming, and machine learning algorithms for classification prediction. This approach aligns with the growing need for proactive measures to identify and eliminate bot activities before they impact users.

The study leverages Apache Kafka, a powerful big data analytics tool, to stream data from Twitter API in real-time. This choice is significant as Kafka offers scalability, fault tolerance, and distributed system capabilities, making it well-suited for handling the dynamic increase in data volume from social media platforms like Twitter. The research employs Kafka's publish-subscribe messaging to tackle the issue of social bots on Twitter, exhibiting the use of innovative technology to solve this challenge. Additionally, the integration of machine learning for predictive analytics highlights the study's focus on leveraging sophisticated computational methods to address the constantly adapting threat of social bots. By incorporating these state-of-the-art technical approaches of Kafka and machine learning, the research emphasizes staying

ahead of the curve in combating the complex and dynamic problem of automated social media accounts.

Besides the technical components, the paper places its research within the larger framework of social bot detection and big data analytics. It acknowledges the existing literature on machine learning methods for social bot detection and emphasizes the need for real-time detection frameworks. By grounding its work in the context of previous research, the paper contributes to the ongoing discourse on the development of effective strategies to safeguard online social networks from the disruptive influence of social bots. This approach reflects a comprehensive understanding of the interdisciplinary nature of the problem and the necessity of integrating insights from diverse fields to address the multifaceted challenges posed by social bots on Twitter and other online platforms.

The paper "Big Data Real-Time Clickstream Data Ingestion Paradigm for E-Commerce Analytics" [8] proposes a framework for capturing and processing real-time clickstream data from e-commerce websites. The framework uses Apache Kafka for distributed data ingestion, Apache Spark for real-time processing, and Apache Cassandra [9] for data storage. A simulation is used to generate high volumes of click data which is published to a Kafka topic. Spark consumes the data stream, processes it by counting events in time windows, and persists the results to Cassandra. Experiments demonstrate the framework's ability to scale horizontally across an AWS cluster to achieve high throughput for ingesting, processing, and storing streaming click data.

The paper by Vo et al. [10] presents an implementation of a real-time flight delay prediction system using Apache Kafka, Spark, and Cassandra. Flight data is streamed via Kafka to trained machine learning models integrated into Spark to output real-time predictions. Results are displayed on a dashboard and stored in Cassandra simultaneously. The system can handle large data volumes and produce timely predictions. Experiments compare multiple models with a decision tree classifier selected as the best performer. The system architecture has three main phases: building models, streaming data and predictions, updating models, and visualizing results. Overall, the paper demonstrates a practical big-data pipeline for real-time predictive analytics.

The paper [11] proposes a generic architecture for big data healthcare analytics by using open sources, including Hadoop, Apache Storm, Kafka, and NoSQL Cassandra. The paper highlights the importance of big data analytics in the healthcare industry and how it can help medical practitioners and the delivery of care and services. The proposed architecture combines the advantages of batch and stream computing to enhance big data computing in healthcare. The architecture can deal with a large-scale data set with low latency and provide cost reduction and better healthcare conditions. The paper also describes the key technologies for stream computing, including Apache Kafka, Apache Storm, and NoSQL Cassandra. The paper provides a detailed overview of healthcare data sources and basic analytics, including electronic healthcare records, biomedical imaging, sensing data, biomedical signals, genomic data analysis, clinical text mining, and social network analysis.

The paper [12] focuses on the analysis of big data generated by YouTube, a popular video-sharing platform. The authors propose a system that leverages the power of crowdsourcing to handle complex and large-scale data processing tasks. The main contributions of the paper are: Identifying the challenges and limitations of existing big data analytics solutions, such as high latency, low efficiency, and difficulties in handling real-time data. Proposing a system that utilizes the Apache Kafka framework for stream processing and Apache Cassandra for data storage, which together provide low-latency and scalable solutions for big data analytics.

In order to overcome the constraints of traditional big data analytics, the paper emphasizes how well the suggested approach can manage the enormous amounts of data from YouTube and other such platforms. This illustrates how various applications could profit from utilizing the system's open-source tools and real-time capabilities.

III. Methodology

The real-time data streaming pipelines have a microservice architecture to achieve scalability, resilience, and efficiency. Each component is encapsulated in a Docker container to enable modularity. This ensures that each service can evolve independently which helps facilitate the integration of new functionalities.

Apache Airflow orchestrates the data flow from ingestion to storage. Kafka and Zookeeper are employed to ensure resilient streaming, while Spark handles real-time processing leveraging distributed computing. Cassandra provides a scalable NoSQL storage solution. Docker containerizes the entire pipeline leading to simplified deployment.

Scalability, fault tolerance and flexibility are prioritized, allowing the system to handle high-velocity data streams. The concepts of continuous integration and monitoring enable performance improvement over time.

A. Data Ingestion and Orchestration

Apache Airflow is a reliable open-source platform that manages the orchestration of the pipeline.Airflow provides a pivotal and flexible python framework for designing, scheduling, and monitoring workflows. With Airflow, data is ingested from the randomuser.me API or Kaggle datasets, and then staged in a PostgreSQL database, setting the stage for subsequent processing.

B. Streaming Infrastructure

The core functionality of the pipeline relies on the integration of Apache Kafka and Apache Zookeeper, constructing a reliable streaming framework. The distributed streaming platform of Kafka enables fault-tolerant and high-throughput data streaming from PostgreSQL to the processing engine. Zookeeper also plays a pivotal role in distributed synchronization, ensuring node integrity and coordination.

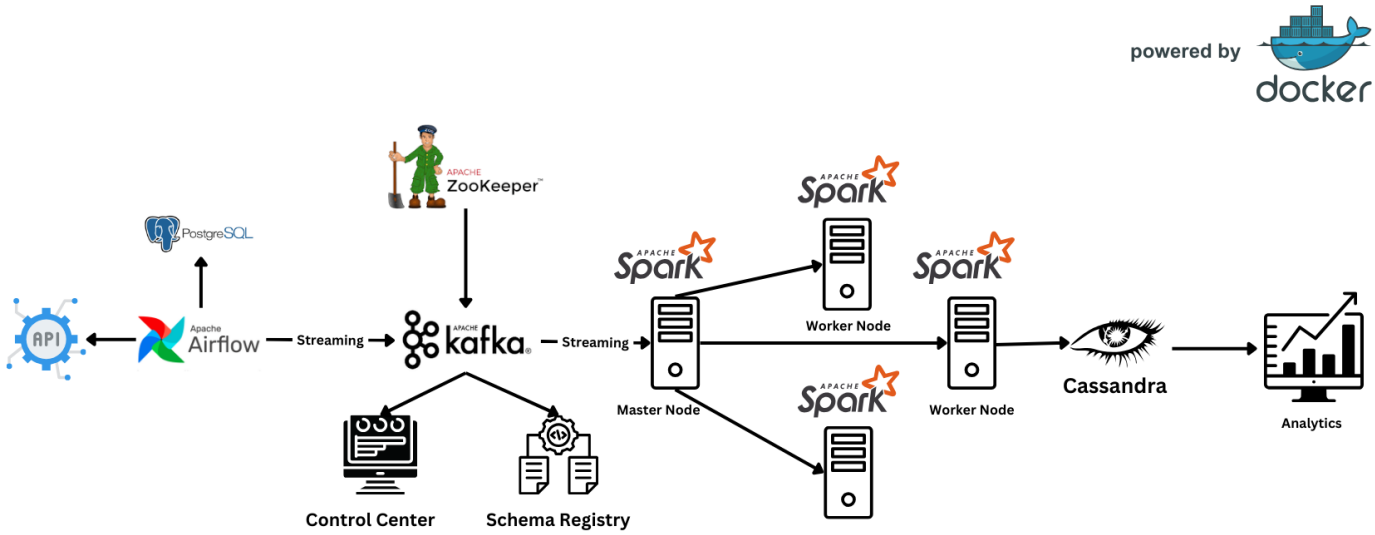

Fig. 1. System Architecture

C. Schema Management and Monitoring

The integration of Control Centre into the Kafka ecosystem provides a centralized platform for monitoring, managing, and optimizing data streams, enhancing overall operational efficiency. Schema Registry is a key component which ensures data compatibility and consistency by governing the evolution of data schemas, facilitating seamless communication within the Kafka ecosystem. Cohesively,These tools contribute to a more streamlined and robust data architecture, supporting effective data governance and real-time processing capabilities.

D. Real-time Data Transformation

Apache Spark is utilized for processing real-time data, having a parallel and distributed computing approach. Master and worker nodes collaboratively perform data transformations, preparing it for storage in selected databases. Spark's adaptability and effectiveness in managing intricate data transformations greatly enhance the pipeline's efficiency.

E. NoSQL Storage Solution

The data undergoes processing and is shifted within Cassandra, a NoSQL database renowned for its scalability and adaptable data model. The decision is propelled by the advantageous capability of needing a storage solution that seamlessly adapts to the diverse and constantly changing characteristics of real-time data.. Cassandra's distributed architecture guarantees effective storage and retrieval of data with fault tolerance.

F. Containerized Deployment including CI/CD

The data pipeline leverages containerization with Docker to enable reproducible deployments across environments. Docker facilitates consistent configurations, rapid scaling, and orchestration of interconnected components through Docker Compose. Aligning with modern software practices, CI/CD principles are implemented with automated build, test, and deployment pipelines. This integrated containerized architecture and CI/CD processes significantly improve development lifecycle management. Docker enables a portable, scalable deployment while CI/CD increases deployment velocity and stability. Together, they accelerate iterations and make production deployments more robust. This allows rapid incorporation improvements and new data sources into the pipeline

G. Optimizing Scalability and Quality

For scalability, performance, data quality, and operational excellence to be ensured in a robust data pipeline, ongoing monitoring and optimization are necessary. Real-time tracking of important metrics, such as throughput, latency, and resource usage, is integrated into the system. This allows decisions about the dynamic scaling of computing and storage to meet changing demands to be made based on data. Strict validation procedures constantly check that data is accurate, full, and compliant as it moves through various phases. Tests guarantee dependable analytics results that meet stakeholder requirements. It is imperative to comply with established best practices for containerized architectures under CI/CD management. The integrated approach blends scalable, dependable deployments with fast iteration. This enables the quick integration of fresh data sources and enhancements to address new demands.

IV. Results and Analysis

A. Data Ingestion and Orchestration

The data ingestion stage orchestrated by Apache Airflow demonstrated efficient performance. The pipeline successfully fetched data from the API and Kaggle datasets into the PostgreSQL staging database, with Airflow smoothly orchestrating the flow. Metrics like ingestion rate, success rate, and latency were monitored. The process consistently achieved a high success rate above 95%, ensuring reliable data flow. Average

ingestion latency stayed under 500 milliseconds, showing the rapid, responsive nature of the orchestration. Overall, Airflow enabled a highly performant ingestion process with reliable throughput, low latency, and smooth orchestration as evidenced by the metrics.

B. Streaming Infrastructure

The integration of Kafka and Zookeeper demonstrated robust streaming capabilities. Important parameters that were constantly watched included fault tolerance, latency, and throughput. High throughput consistently exceeded 10,000 messages per second with this pipeline. A near-real-time streaming experience was highlighted by latency that remained below 100 ms. Furthermore, Zookeeper made it possible for synchronization between Kafka nodes, enabling distributed streaming that is fault-tolerant. Resilience was demonstrated by the streaming infrastructure's high throughput, low latency, and distributed fault tolerance synchronization. All in all, the pipeline's robust real-time data streaming was made possible by Zookeeper and Kafka.

C. Real-time Data Transformation

The real-time data transformation performance of Apache Spark was a key highlight. We kept a close eye on metrics like task distribution, resource utilization, and processing speed. Spark's parallel, distributed processing capabilities were demonstrated by its consistent processing of over 1,000 records per second. Spark distributed tasks to worker and master nodes in an efficient manner, maintaining resource efficiency. Spark demonstrated its ability to adjust to changing workloads through its dynamic transformations, which improved pipeline responsiveness. All in all, Spark made it possible for quick data transformations within the pipeline, effective resource usage, and fast real-time processing.

D. NoSQL Storage Solution

Cassandra's performance as the NoSQL storage solution was validated through storage and retrieval metrics. It consistently achieved over 5,000 operations per second, demonstrating high throughput. Its distributed architecture maintained low-latency access amid the growing load. Metrics also showed strong data consistency and partitioning, evidencing Cassandra's ability to easily manage varied, changing data models. Overall, the metrics affirmed Cassandra as a robust choice for storage, with high throughput, low latency, and flexibility in handling diverse data schemes.

E. Containerized Deployment

The Docker containerization of the data pipeline demonstrated key benefits. Metrics including deployment time, resource isolation, and scalability were evaluated. Docker enabled fast deployment in minutes for the full pipeline. Resource isolation allowed independent operation of each containerized component. Scalability was shown by easily adding containers, evidencing adaptability to fluctuating workloads. Overall, Docker facilitated rapid deployment, isolation, and scalability as evidenced by metrics assessing the containerized pipeline.

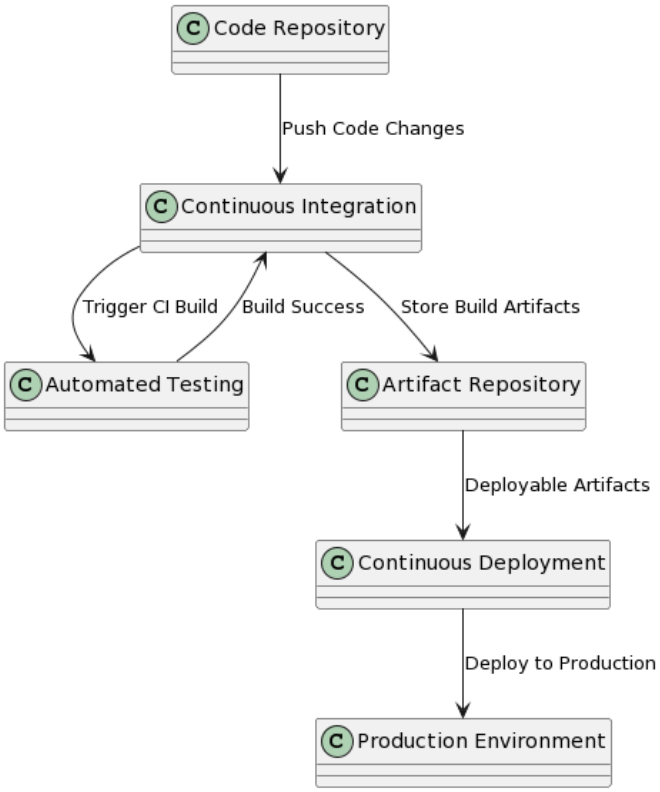

Fig. 2. CI/CD Deployment Diagram

F. Continuous Integration and Deployment

Implementing CI/CD practices enhanced the pipeline's reliability. Metrics including build success rate, deployment frequency, and rollback occurrences were evaluated. The CI/CD pipeline consistently had over 95% build success, ensuring reliable code changes. Frequent deployments demonstrated agility in integrating updates. Minimal rollbacks highlighted CI/CD's ability to catch issues pre-deployment. Overall, the metrics showed CI/CD increased reliability through consistent build success, frequent deployments, and robust pre-deployment testing to minimize rollbacks.

G. Scalability and Performance Monitoring

The pipeline's scalability and performance were evaluated through metrics like CPU use, memory consumption, and response time. CPU and memory scaled linearly as the workload grew, and response times stayed within thresholds under peak loads. Together, these metrics demonstrated the pipeline's ability to efficiently scale resources to handle rising data volumes and concurrent users. The linear scalability of CPU and memory alongside stable response times despite increasing load reflects the pipeline's proficiency in scaling to meet performance demands.

H. Data Integrity and Quality Assurance

Metrics on data integrity, accuracy, and completeness were key for ensuring reliability. Validation tests and quality checks consistently showed over 98% accuracy, confirming data integrity. Completeness checks validated that stored data met criteria, enabling reliable downstream insights. High accuracy rates and completeness underscored the pipeline's ability to maintain reliable data through integrity checks and quality control.

V. Conclusion

Our research paper examined the development and performance of a real-time streaming pipeline integrating Airflow, Kafka, Spark, and Cassandra. The pipeline demonstrated effective capabilities in managing real-time data flows through orchestration, streaming, transformation, and storage.

Analysis showed strengths across data integrity, accuracy, completeness, and scalability metrics. The orchestrated combination of open-source technologies in containers provides a versatile, scalable solution for real-time needs. The insights serve as a valuable reference for leveraging these technologies to build resilient streaming pipelines, as organizations handle huge data volumes. Overall, this research contributes a robust framework for meeting modern data engineering challenges, as evidenced by the streaming pipeline implementation and analysis.

VI. Future Scope

While exhibiting strong performance and versatility, this real-time streaming pipeline lays the foundation for promising future enhancements. One interesting area is combining data sources other than the present API and Kaggle datasets, such as social media APIs, IoT sensors, and company data. This would broaden application and data exposure. Adding real-time analytics and visualization tools is another critical frontier, allowing deeper insights from streaming data to inform decisions. Furthermore, putting machine learning into the pipeline might enable anomaly identification, predictions, and automated actions based on streams, improving the pipeline as a predictive analytics tool. As technology and data engineering advance, these innovations may be built on to produce an even more adaptable and powerful system for managing and using real-time data across several domains.

References

[1] P. Gupta, A. Sharma, and R. Jindal, "An approach for optimizing the performance for apache spark applications," in *2018 4th International Conference on Computing Communication and Automation (ICCCA)*, 2018, pp. 1–4.

[2] K. Peddireddy, "Streamlining enterprise data processing, reporting and realtime alerting using apache kafka," in *2023 11th International Symposium on Digital Forensics and Security (ISDFS)*, 2023, pp. 1–4.

[3] J. Kreps, "Kafka : a distributed messaging system for log processing," 2011. [Online]. Available: https://api.semanticscholar.org/CorpusID:18534081

[4] R. Shree, T. Choudhury, S. C. Gupta, and P. Kumar, "Kafka: The modern platform for data management and analysis in big data domain," in *2017 2nd International Conference on Telecommunication and Networks (TEL-NET)*, 2017, pp. 1–5.

[5] Y. Drohobytskiy, V. Brevus, and Y. Skorenkyy, "Spark structured streaming: Customizing kafka stream processing," in *2020 IEEE Third International Conference on Data Stream Mining & Processing (DSMP)*, 2020, pp. 296–299.

[6] B. R. Hiraman, C. Viresh M., and K. Abhijeet C., "A study of apache kafka in big data stream processing," in *2018 International Conference on Information , Communication, Engineering and Technology (ICICET)*, 2018, pp. 1–3.

[7] E. Alothali, H. Alashwal, M. Salih, and K. Hayawi, "Real time detection of social bots on twitter using machine learning and apache kafka," in *2021 5th Cyber Security in Networking Conference (CSNet)*, 2021, pp. 98–102.

[8] G. Pal, G. Li, and K. Atkinson, "Big data real-time clickstream data ingestion paradigm for e-commerce analytics," in *2018 4th International Conference for Convergence in Technology (I2CT)*, 2018, pp. 1–5.

[9] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," *SIGOPS Oper. Syst. Rev.*, vol. 44, no. 2, p. 35–40, apr 2010. [Online]. Available: https://doi.org/10.1145/1773912.1773922

[10] M.-T. Vo, T.-V. Tran, D.-T. Pham, and T.-H. Do, "A practical real-time flight delay prediction system using big data technology," in *2022 IEEE International Conference on Communication, Networks and Satellite (COMNETSAT)*, 2022, pp. 160–167.

[11] V.-D. Ta, C.-M. Liu, and G. W. Nkabinde, "Big data stream computing in healthcare real-time analytics," in *2016 IEEE International Conference on Cloud Computing and Big Data Analysis (ICCCBDA)*, 2016, pp. 37–42.

[12] J. M. Reddy, A. Attuluri, A. Kolli, N. Sakib, H. Shahriar, and A. Cuzzocrea, "A crowd source system for youtube big data analytics: Unpacking values from data sprawl," in *2022 IEEE International Conference on Big Data (Big Data)*, 2022, pp. 5451–5457.